

FULL STACK DEVELOPMENT ESE3014

Lab 7

Learning Outcomes:

1. REST API Endpoints

1. REST API Endpoints

Learning Outcomes:

REST API Endpoints

- i) Sign up and launch Postman
- ii) Sample API Controller
- iii) Test REST API Endpoints
- iv) Create REST API Endpoints

Sign up and launch Postman



Postman is an API platform for building and using APIs. Postman simplifies each step of the API lifecycle and streamlines collaboration so you can create better APIs.

Sign up for the free version.
<https://www.postman.com/>

Download the Postman application for API testing.

Sample API Controller

Dealing with any form of development, you may find yourself in a situation where all your code are error free, but the application does not work. This could be a common experience when developing application that use multiple layers of technology.

The communication between each technology is also a responsibility of the developer. Therefore, it is important to ensure that your application is tested before it leaves its factory. Of course, when this application is deployed in the production environment, you will again need to test it before it goes live.

The .NET Blazor project does not included a sample API controller for our testing. For the sample API controller, we can get it from another project.

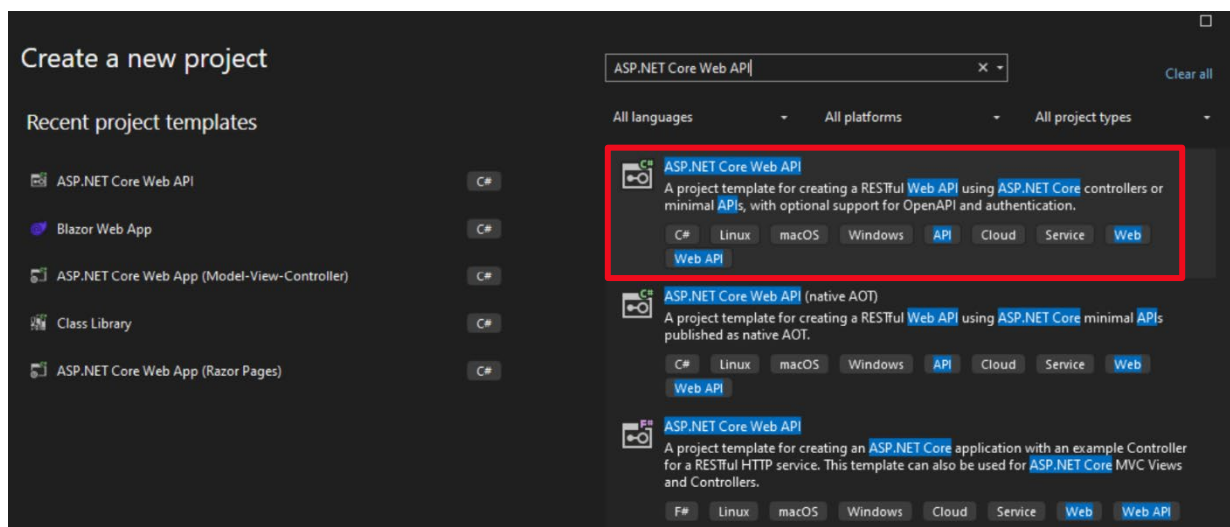
Let's start by learning how to test a REST API and be familiar with the testing tool, Postman.

Practice 112

Create a new project.

Search for **ASP.NET Core Web API**.

Choose the correct project with the C# logo.



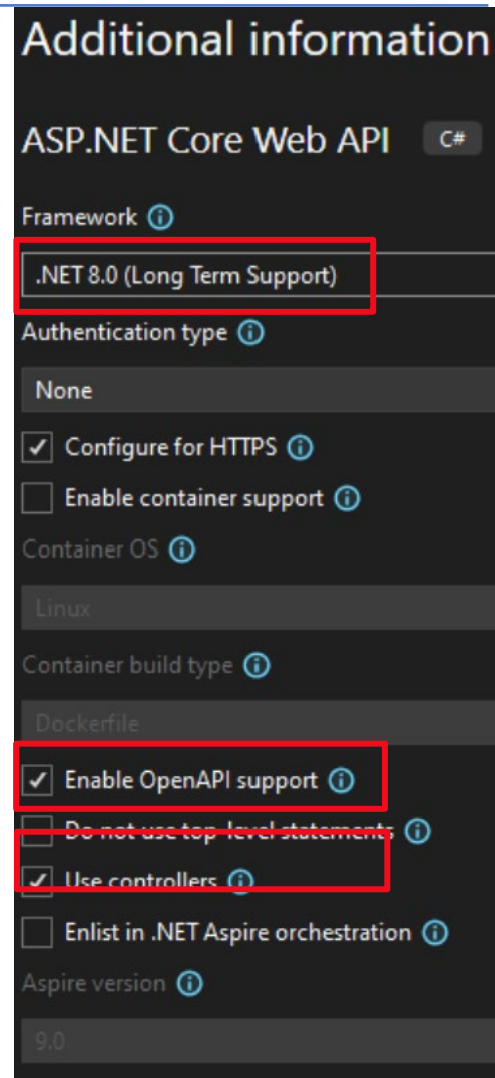
In the Configure your new project dialog, name the project **SampleWebAPI** and select Next.

In the Additional information dialog:
Confirm the Framework is
.NET 8.0 (Long Term Support).

Confirm the checkbox for **Use controllers** is checked.

Confirm the checkbox for
Enable OpenAPI support is checked.

Select Create.



Additional information

ASP.NET Core Web API C#

Framework ⓘ
.NET 8.0 (Long Term Support)

Authentication type ⓘ
None

☒ Configure for HTTPS ⓘ
☐ Enable container support ⓘ

Container OS ⓘ
Linux

Container build type ⓘ
Dockerfile

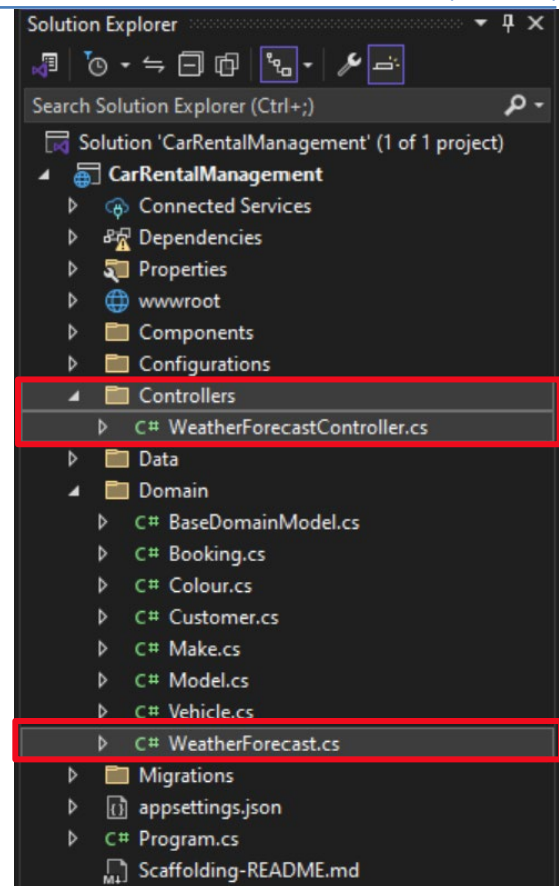
☒ Enable OpenAPI support ⓘ
☐ Do not use top-level statements ⓘ
☒ Use controllers ⓘ

☐ Enlist in .NET Aspire orchestration ⓘ

Aspire version ⓘ
9.0

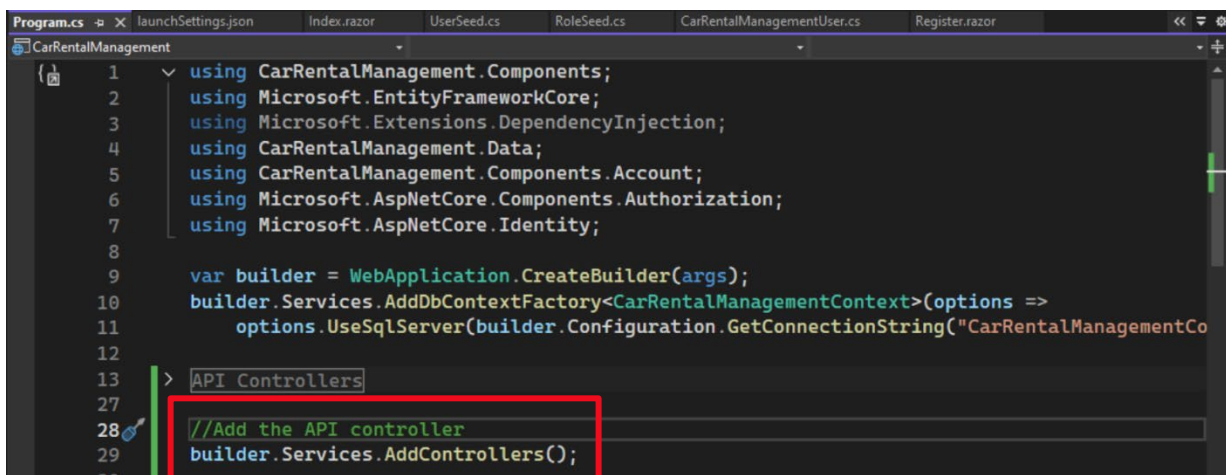
In the solution explorer, **copy** the Controllers folder which include the WeatherForecastController.cs file as well as the WeatherForecast.cs file.

Open your Visual Studio –
CarRentalManagement project, **paste** the Controllers folder in the project root and the WeatherForecast.cs file in the Domain folder.



Open the Program.cs file. Add the controller service to the project. Do note that it has to be after line 9 in the screen capture.

//Add the API controller
builder.Services.AddControllers();



In the same Program.cs file. Add the default controller route to the app. Do note that it has to be after line 62 in the screen capture.

//Add the Default controller route
app.MapDefaultControllerRoute();

```

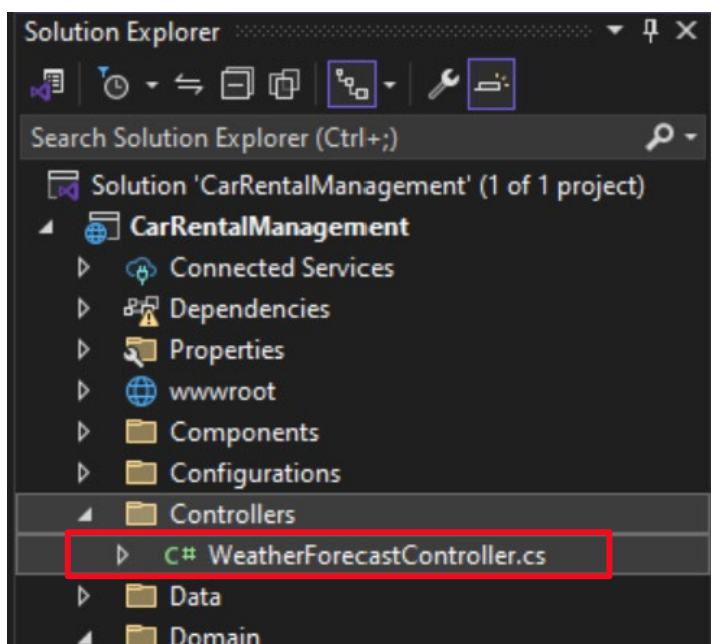
59
60 builder.Services.AddSingleton<IEmailSender<CarRentalManagementUser>, IdentityNoOpEmailSen
61
62 var app = builder.Build();
63
64 #region API Controllers
65 //Add the Default controller route
66 app.MapDefaultControllerRoute();
67 #endregion
68
69 // Configure the HTTP request pipeline.
70 if (!app.Environment.IsDevelopment())
71 {

```

Practice 113

Let's look at the sample REST API. Go to **CarRentalManagement >> Controllers >> WeatherForecastController.cs**.

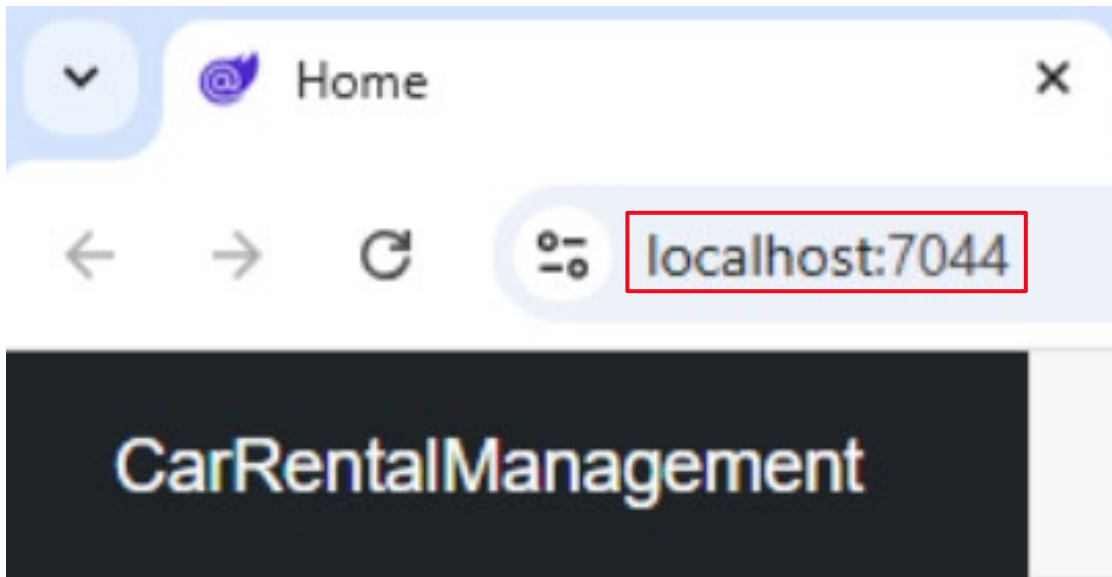
An API controller file is where you can define controller methods. In this sample API, there is only one method, and this method returns a random weather forecast. To call the REST API, you will need to form the URL.



Your server's URL is configured in **CarRentalManagement >> Properties >> launchSettings.json**.

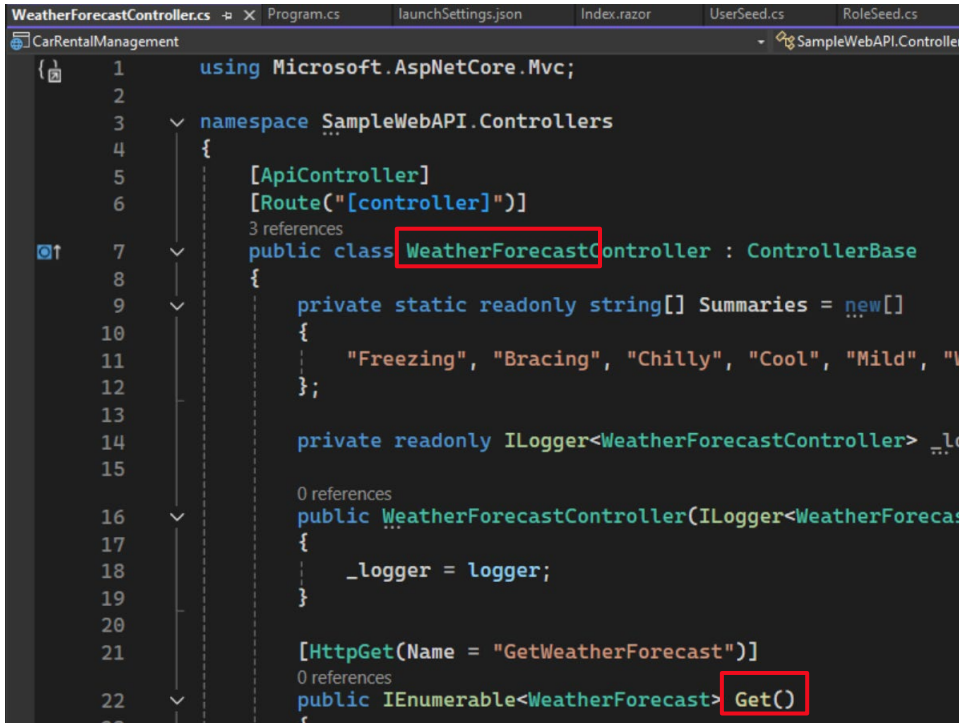
```
"https": {  
  "commandName": "Project",  
  "dotnetRunMessages": true,  
  "launchBrowser": true,  
  "applicationUrl": "https://localhost:7044;http://localhost:5114",  
  "environmentVariables": {  
    "ASPNETCORE_ENVIRONMENT": "Development"  
  }  
},
```

Or you could just click on the **Start without Debugging**.
Your port number may be different.
In this case, the server's url is **https://localhost:44392**



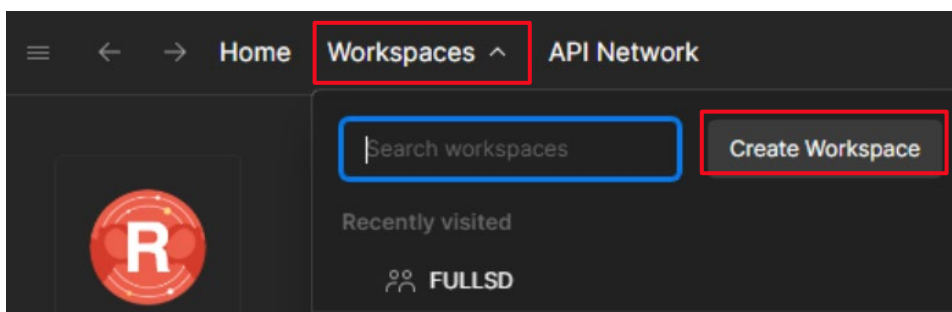
Extend the URL with the controller's name and omit the postfix "Controller". Your controller's URL should be: **https://localhost:44392/weatherforecast**

Take note of the HTTP verb, **GET**.



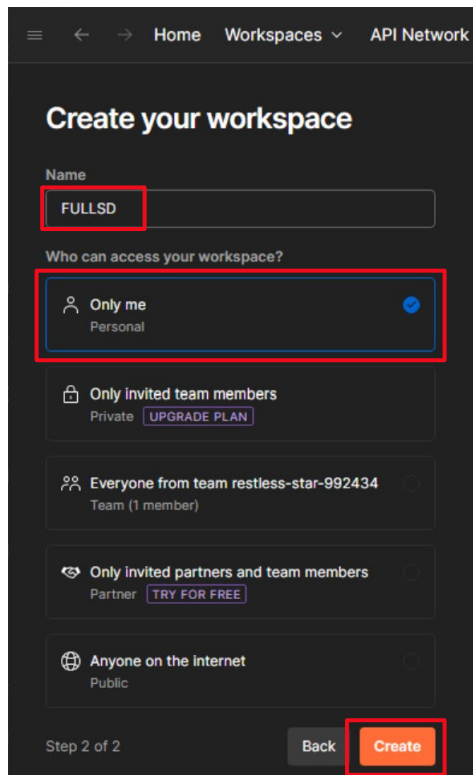
```
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace SampleWebAPI.Controllers
4 {
5     [ApiController]
6     [Route("[controller]")]
7     public class WeatherForecastController : ControllerBase
8     {
9         private static readonly string[] Summaries = new[]
10         {
11             "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
12         };
13
14         private readonly ILogger<WeatherForecastController> _logger;
15
16         public WeatherForecastController(ILogger<WeatherForecastController> logger)
17         {
18             _logger = logger;
19         }
20
21         [HttpGet(Name = "GetWeatherForecast")]
22         public IEnumerable<WeatherForecast> Get()
```

We will transit to the Postman app. After you sign in, select Workspaces and select Create Workspace.



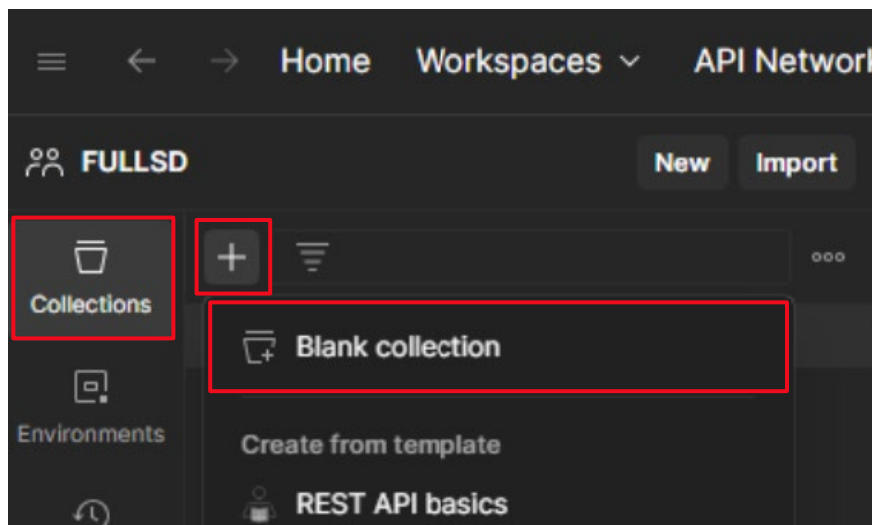
Choose **Blank workspace**.

Type a name for your workspace, select **Only me** and click **Create** button.



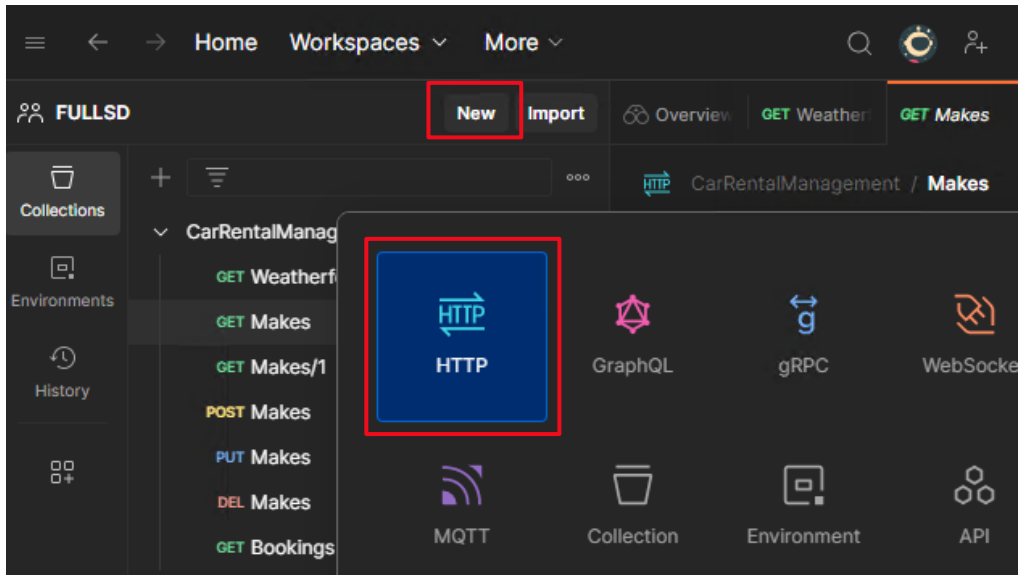
The screenshot shows the 'Create your workspace' form. The 'Name' field contains 'FULLSD'. Under 'Who can access your workspace?', the 'Only me' option is selected. The 'Create' button is highlighted.

In your workspace, create a new collection and by selecting Collection tab and click on the + button. Create a collection called **CarRentalManagement**.

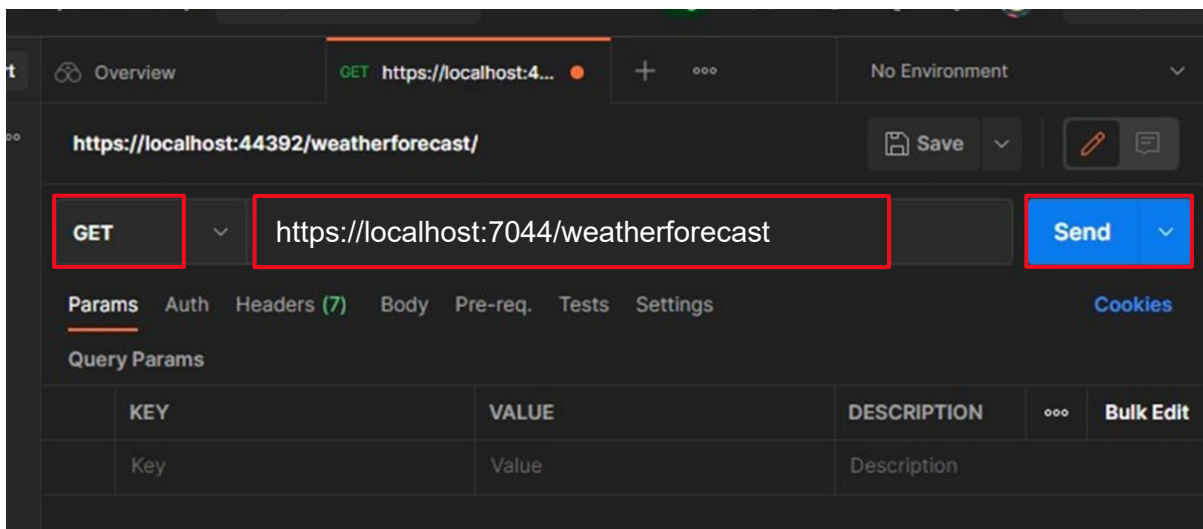


The screenshot shows the workspace interface for 'FULLSD'. The 'Collections' tab is selected. A '+' button is clicked, and a 'Blank collection' is created. The 'Create from template' section shows 'REST API basics'.

Click New and select HTTP Request.



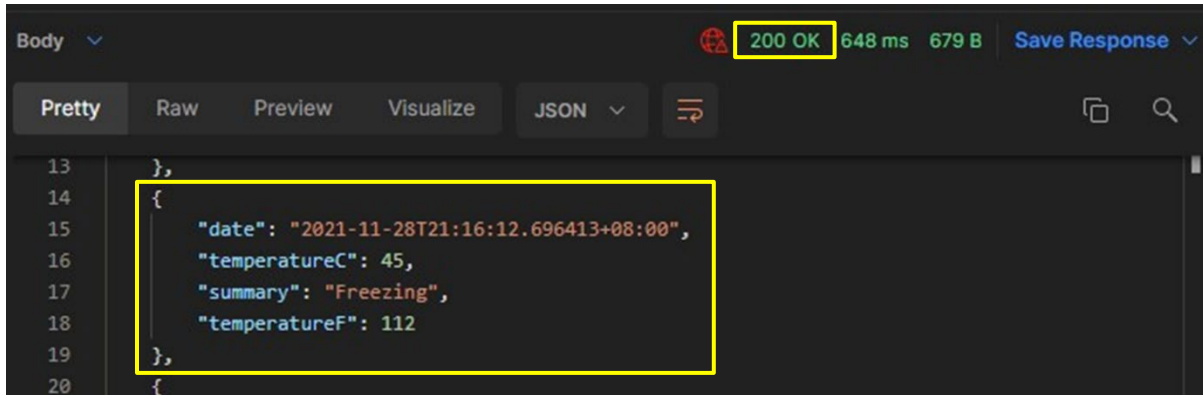
Ensure the HTTP verb is GET. Type in the URL <https://localhost:7044/weatherforecast> and click Send button. Ensure that your CarRentalManagement has Started without Debugging. Ensure your localhost:[Port number] is correct.



The HTTP status code is 200 Ok. The request has succeeded. For more information about HTTP status codes: <https://www.restapitutorial.com/httpstatuscodes.html>

The information returned with the response is dependent on the method used in the request, for example: GET an entity corresponding to the requested resource is sent in the response. The requested data is displayed in JSON.

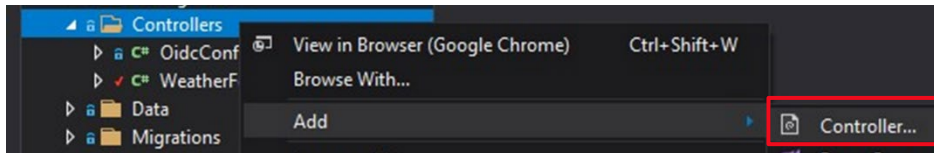
https://www.w3schools.com/js/js_json_intro.asp



```
Body 200 OK 648 ms 679 B Save Response
Pretty Raw Preview Visualize JSON
13 },
14 {
15   "date": "2021-11-28T21:16:12.696413+08:00",
16   "temperatureC": 45,
17   "summary": "Freezing",
18   "temperatureF": 112
19 },
20 {
```

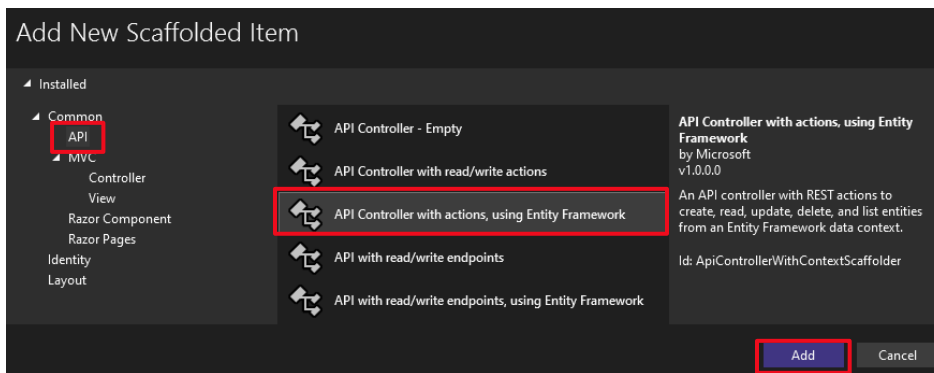
Create REST API Endpoints

Practice 114

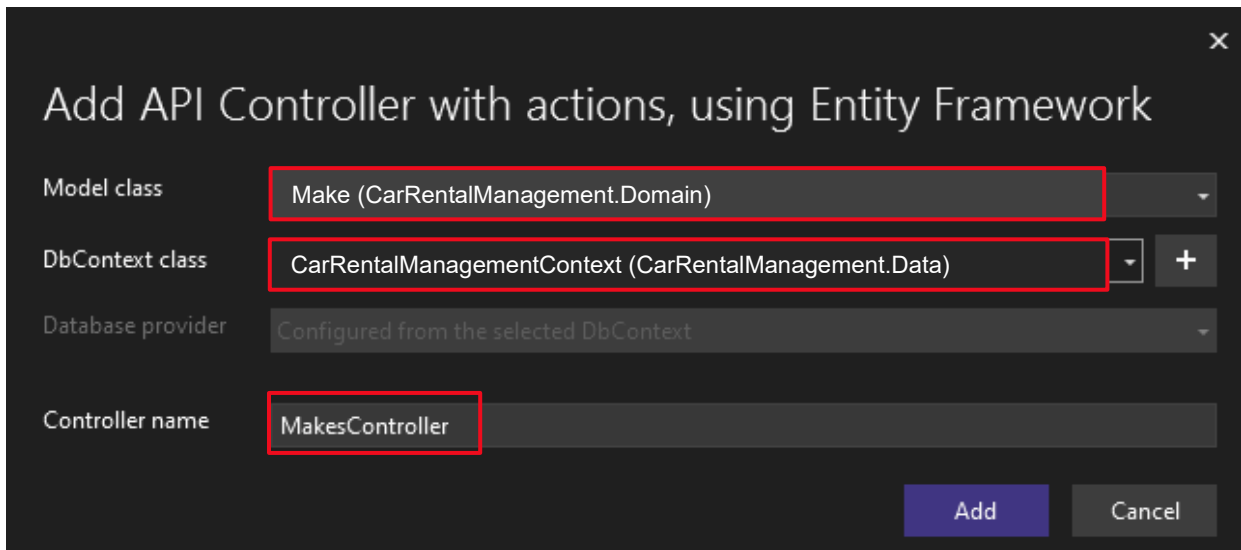


The benefit of using .NET Core is that it allows scaffolding. Right-click on controller folder, select add and select Controller.

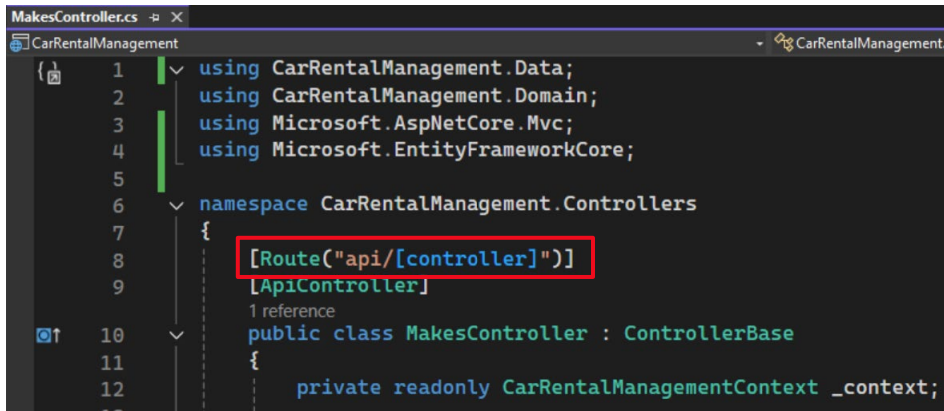
In the dialog box, choose **API**, and select **API Controller with actions, using Entity Framework**. Click Add button.



Select the **Make** model class, the **CarRentalManagementContext** data context class and leave the default controller name as **MakesController**. Click Add button to start the scaffolding.



Once the MakesController is created and we can start testing it with Postman app. The route attribute **[Route("api/[controller]")]** has the additional api/ before the [controller]. The URL is now **https://localhost:7044/api/makes**

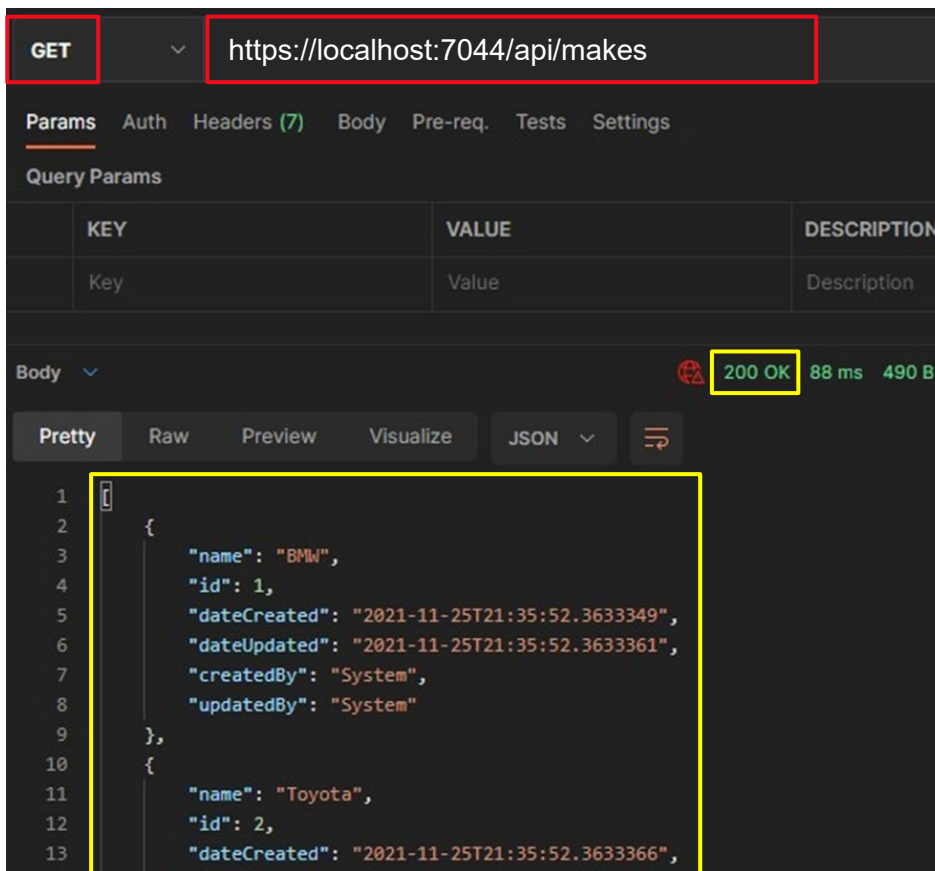


```

1  using CarRentalManagement.Data;
2  using CarRentalManagement.Domain;
3  using Microsoft.AspNetCore.Mvc;
4  using Microsoft.EntityFrameworkCore;
5
6  namespace CarRentalManagement.Controllers
7  {
8      [Route("api/[controller]")]
9      [ApiController]
10     public class MakesController : ControllerBase
11     {
12         private readonly CarRentalManagementContext _context;
    
```

Practice 115

For makes controller, the **GET** method should return status 200 and all the make data. **Remember to save your API request on Postman so that you can reuse the request.**



GET **https://localhost:7044/api/makes**

Params Auth Headers (7) Body Pre-req. Tests Settings

Query Params

KEY	VALUE	DESCRIPTION
Key	Value	Description

Body **200 OK** 88 ms 490 B

Pretty Raw Preview Visualize JSON

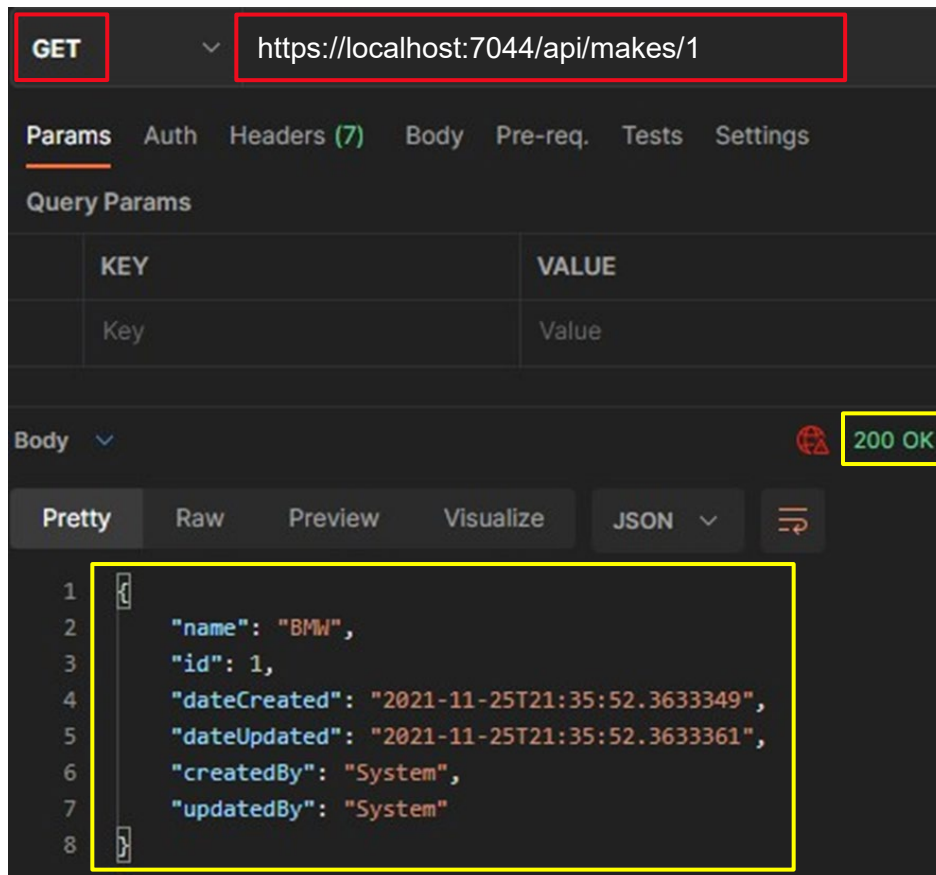
```

1  {
2    {
3      "name": "BMW",
4      "id": 1,
5      "dateCreated": "2021-11-25T21:35:52.3633349",
6      "dateUpdated": "2021-11-25T21:35:52.3633361",
7      "createdBy": "System",
8      "updatedBy": "System"
9    },
10   {
11     "name": "Toyota",
12     "id": 2,
13     "dateCreated": "2021-11-25T21:35:52.3633366",
    
```

Practice 116

Update the request URL to <https://localhost:7044/api/makes/1>

For makes controller, the **GET by Id** method should return status 200 and only the make data with Id = 1.



Practice 117

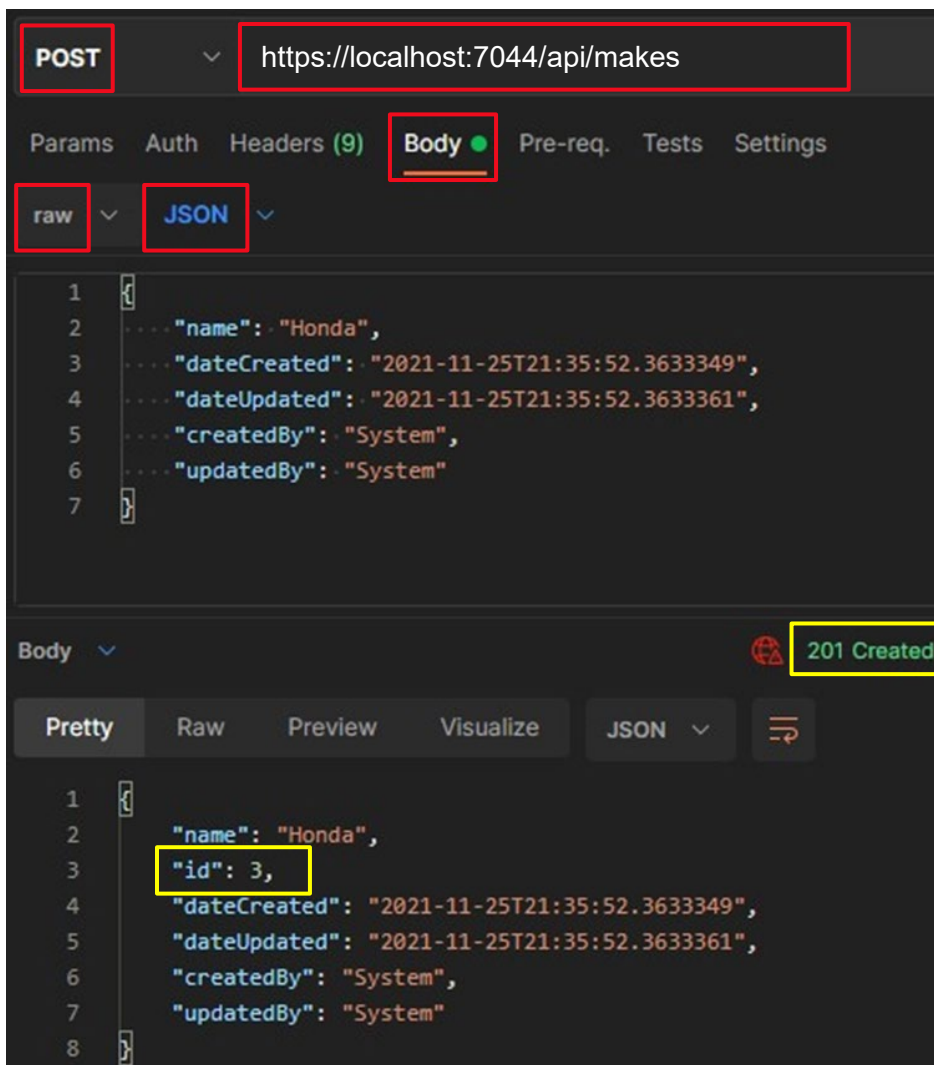
Change the verb to **POST**. For makes controller, the **POST** method should return status 201 Created and the new make data with a new Id.

Update the request URL to <https://localhost:7044/api/makes>

Select **Body**, select **Raw** and select **JSON**.

Copy the results from the previous request and paste.

Change name to Honda and delete id.



Practice 118

Change the verb to **PUT**. For makes controller, the **PUT** method should return status 204 No Content and there is no return data displayed.

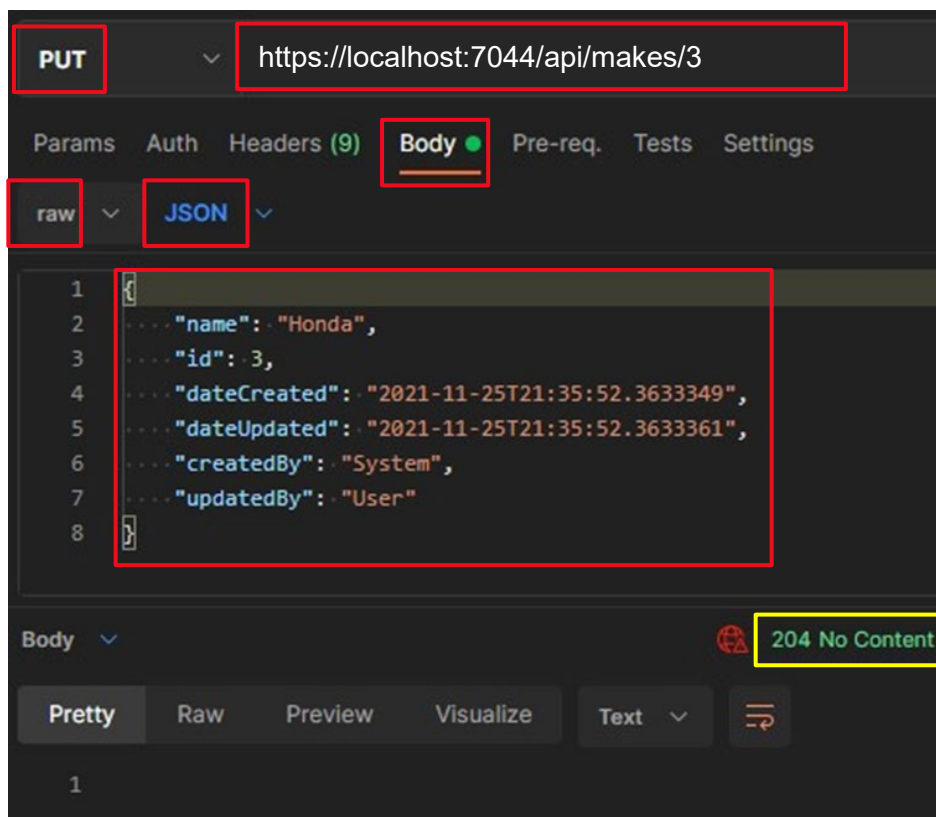
Update the request URL to <https://localhost:7044/api/makes/3>

Select **Body**, select **Raw** and select **JSON**.

Copy the results from the previous request and paste.

Change `updatedBy` to `User` and put back the `"id": 3`,

Use GET by Id to see updated data.



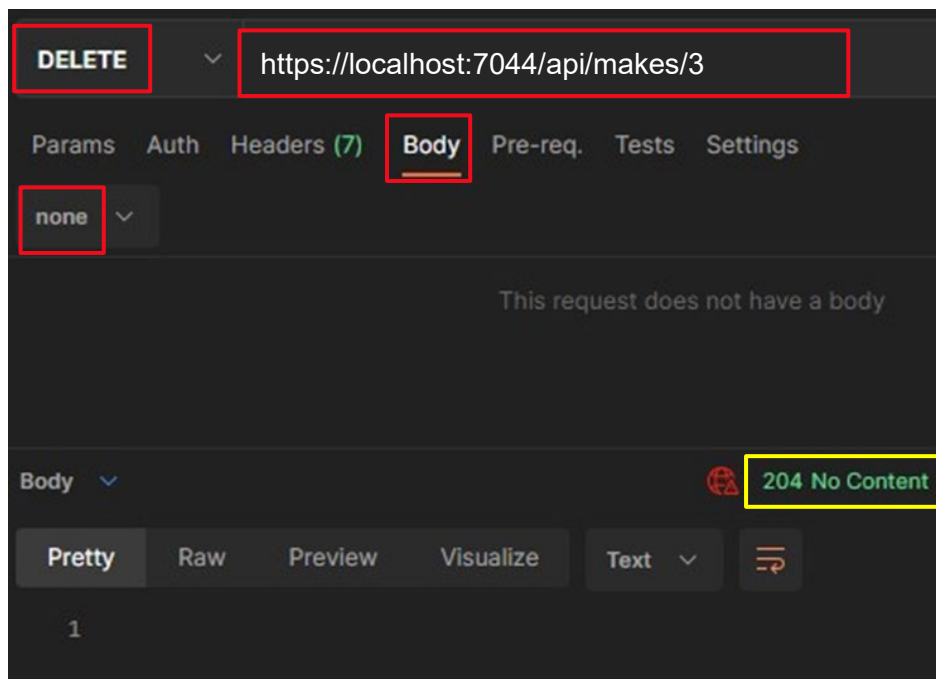
Practice 119

Change the verb to **DELETE**. For makes controller, the **DELETE** method should return status 204 No Content and there is no return data displayed.

Update the request URL to <https://localhost:7044/api/makes/3>

Select **Body** and select **None**.

Use **GET** to list all data and see if data has been deleted.



Exercise 33

Submit the full screenshot of all the Postman requests you have created and ensure that your API requests for Makes are captured in your screenshot.

- i. GET
- ii. GET (By Id)
- iii. POST
- iv. PUT
- v. DELETE

